Laboratório Prático: Redes e Containers com Docker

Isolamento de Serviços e Roteamento — Mini Radar ENEM

Unidade Curricular: Computação em Nuvem

Atividade: Aula 11 — Laboratório hands-on + Incremento 4

Projeto base: Mini Radar ENEM

Entregável AVA — Evidências + Relatório Técnico

Sumário dos entregáveis

#	Entregável	Onde está neste PDF
1	Inspeção da rede interna (`backend-net`): CIDR, `Internal: true` e IPs	Evidência 1
2	Testes de isolamento (Passo 5): proxy OK, proxy db, backend db	Evidência 2
3	Os 4 desafios investigativos (Passo 6)	Evidência 3
4	Diagrama arquitetural da VPC `10.20.0.0/16` (Incremento 4)	Evidência 4
5	Relatório técnico de análise e defesa arquitetural	Seção Relatório Técnico

Arquitetura validada no laboratório

```
          Internet / Host
                |
           :8080 (unica porta publicada)
                |
  +-------------- radar-proxy (172.20.10.10) --------------+
  |             frontend-net  172.20.10.0/24               |
  |                    |                       |
  |             radar-backend (172.20.10.20)               |  <- multi-homed
  +--------------------------------------------------------+
                       |  (172.20.20.20)
  +------------- backend-net 172.20.20.0/24 (internal) ----+
  |                    |                       |
  |                radar-db (172.20.20.50, PostgreSQL)     |  <- sem porta no host
  +--------------------------------------------------------+
```

Ambiente no ar — `docker compose up -d` : 3 containers (proxy, backend, db) e 2 redes (frontend-net e backend-net `internal`). Apenas `radar-proxy` publica a porta `8080` .

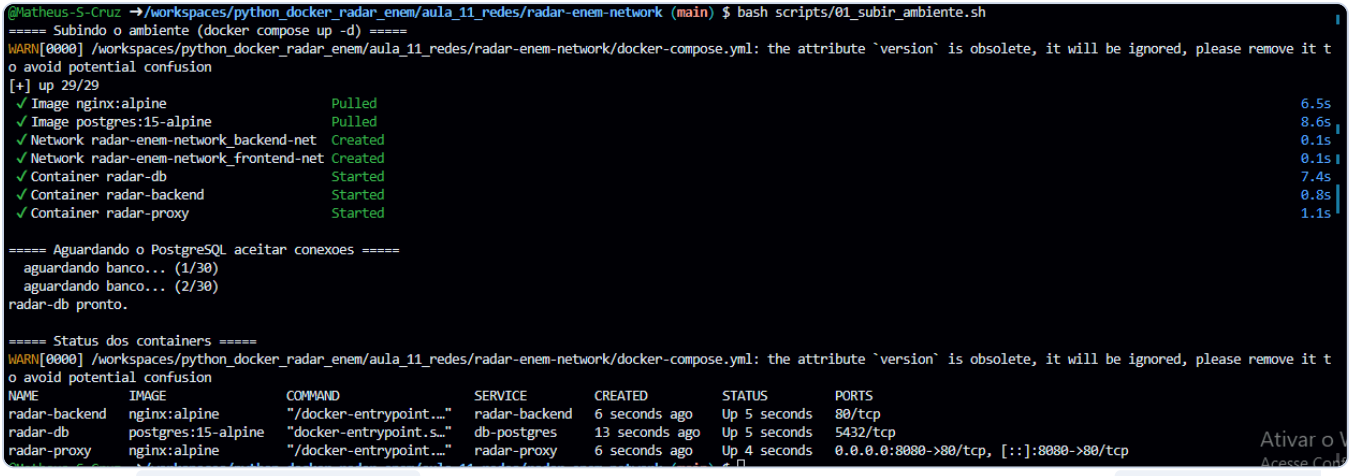


Figura 0 — `docker compose up -d` : imagens, redes e containers criados; PostgreSQL pronto; `docker compose ps` .

Evidência 1 — Inspeção da rede interna

Saída de `docker network inspect radar-enem-network_backend-net` comprovando o bloco CIDR `172.20.20.0/24`, o status `"Internal": true` e os IPs atribuídos a `radar-backend` (`.20`) e `radar-db` (`.50`).

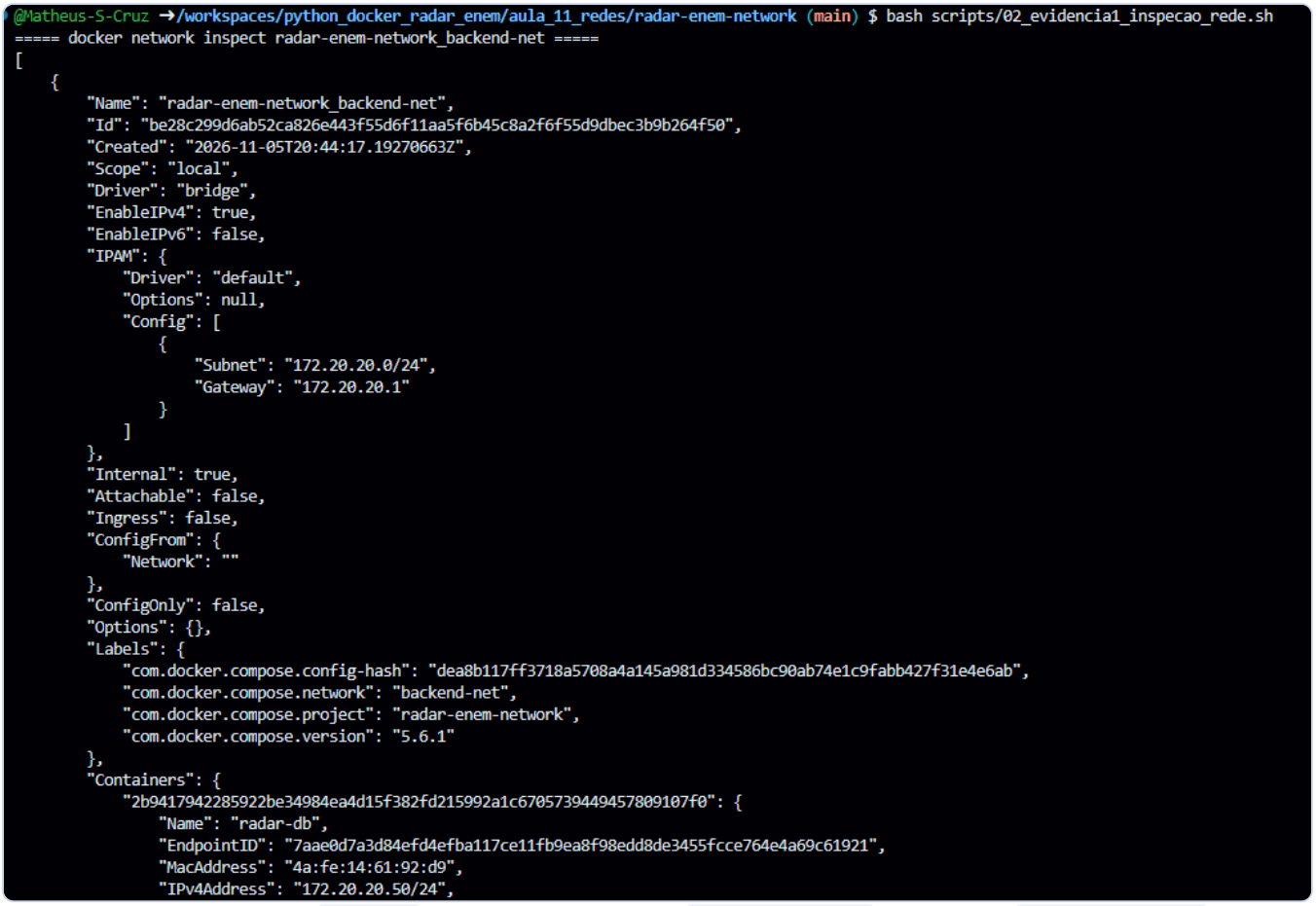


Figura 1.1 — Cabeçalho do `inspect`: driver bridge, Subnet `172.20.20.0/24`, Gateway e `"Internal": true`.

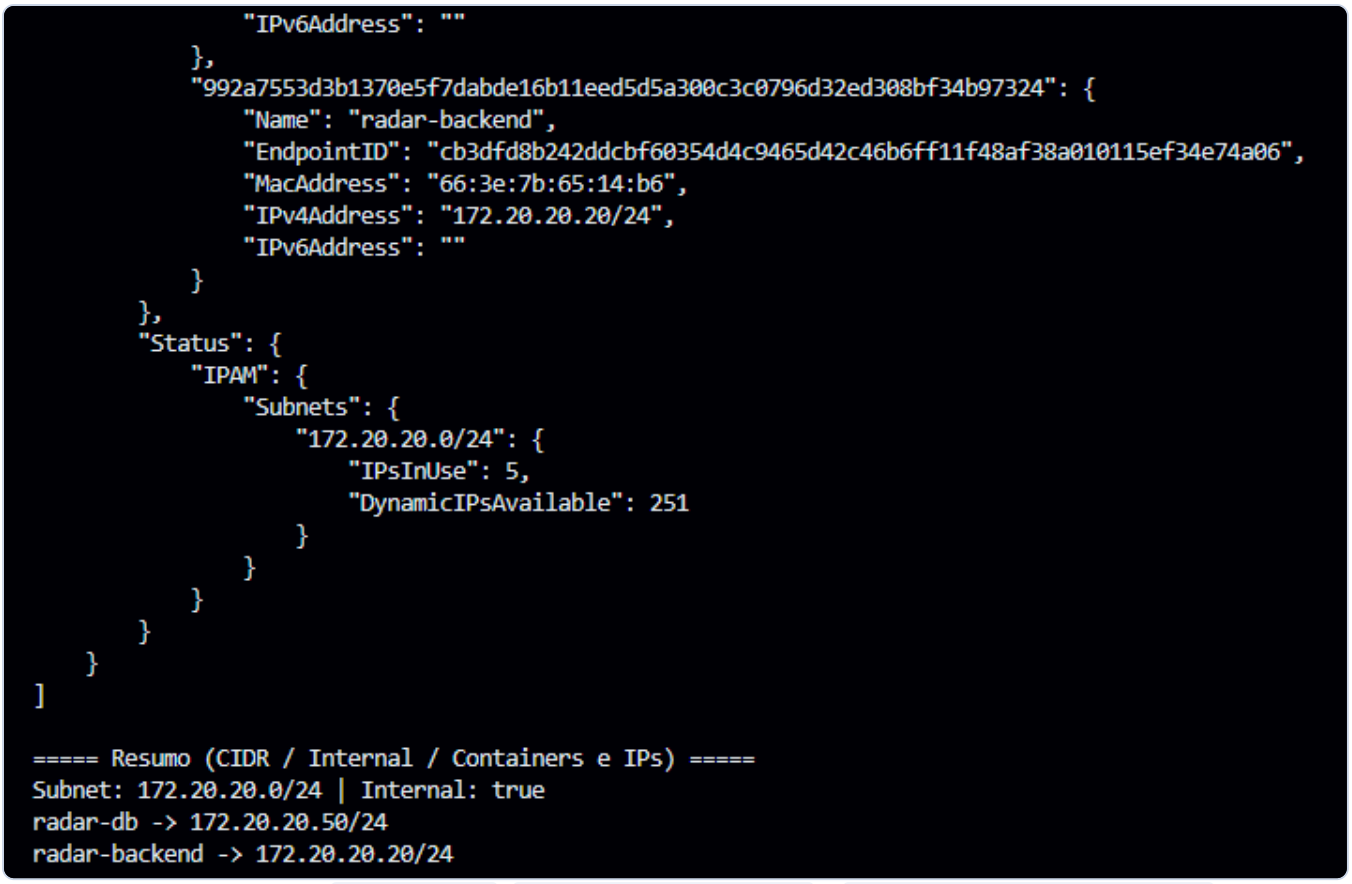


Figura 1.2 — Containers na `backend-net`: `radar-db 172.20.20.50` e `radar-backend 172.20.20.20`.

Evidência 2 — Testes de isolamento (Passo 5)

Demonstra, na ordem: (a) o proxy público respondendo `200 OK` em `localhost:8080`; (b) a **falha** de `radar-proxy` → `radar-db` (o proxy não resolve nem alcança o banco); e (c) o **sucesso** de `radar-backend` → `radar-db` (o backend é multi-homed e participa da rede interna).

> A descoberta de serviço é provada pela conexão real via **nome** do container (`nc -z -v radar-db 5432`): se conecta, o nome resolveu pelo DNS do Docker (`127.0.0.11`) e existe rota; se não, está isolado.

```
@Matheus-S-Cruz ➜/workspaces/python_docker_radar_enem/aula_11_redes/radar-enem-network (main) $ bash scripts/03_evidencia2_isolamento.sh
===== (a) Teste 1: Acesso externo via Proxy (localhost:8080) =====
HTTP/1.1 200 OK
Server: nginx/1.31.8
Date: Thu, 05 Nov 2026 21:35:03 GMT
Content-Type: text/html
Content-Length: 81
Connection: keep-alive
Last-Modified: Thu, 05 Nov 2026 21:32:59 GMT
ETag: "6aecf60b-51"
Accept-Ranges: bytes
X-Proxy-Layer: frontend-net-dmz

{"aplicacao":"Mini Radar ENEM","status":"ONLINE","banco_privado":"172.20.20.50"}

===== (b) Teste 2: radar-proxy -> radar-db  (DEVE FALHAR) =====
--- nc -z -v -w5 radar-db 5432 a partir do proxy (resolucao+conexao pelo NOME) ---
nc: bad address 'radar-db'
>>> Como esperado: o proxy NAO descobre nem alcanca radar-db
>>> (nome fora do escopo da backend-net e sem rota ate a rede interna).

===== (c) Teste 3: radar-backend -> radar-db  (DEVE FUNCIONAR) =====
--- nc -z -v -w5 radar-db 5432 a partir do backend (resolucao+conexao pelo NOME) ---
radar-db (172.20.20.50:5432) open
>>> SUCESSO: backend descobre radar-db pelo nome e conecta na porta 5432
>>> (backend e multi-homed: participa da backend-net).
```

Figura 2 — (a) 200 OK + JSON; (b) `nc: bad address 'radar-db'` → proxy isolado do banco; (c) `radar-db (172.20.20.50:5432) open` → backend conecta.

Evidência 3 — Os 4 desafios investigativos (Passo 6)

Desafio 1 — Escopo do DNS interno

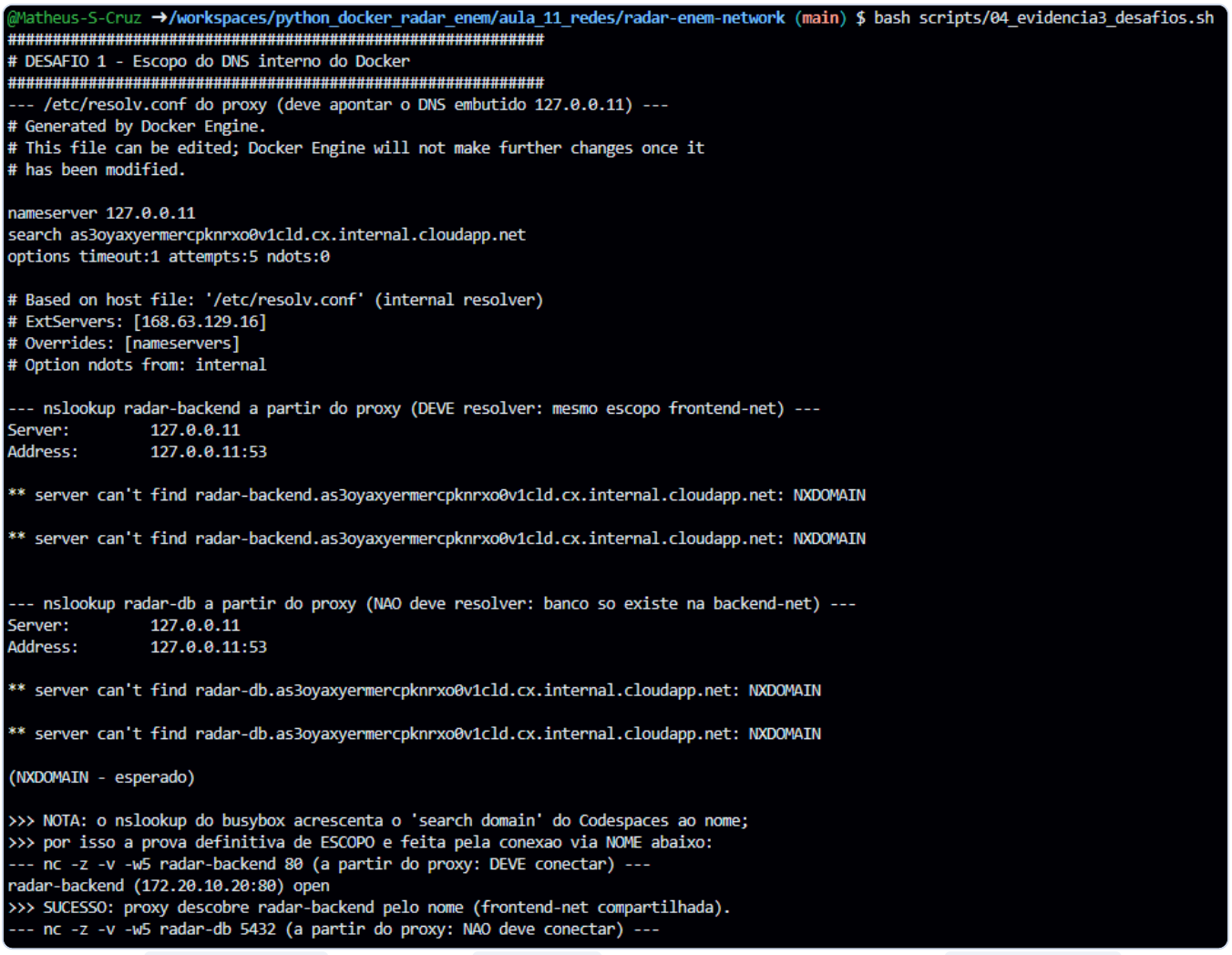


Figura 3.1 — `/etc/resolv.conf` aponta para `127.0.0.11`; prova de escopo por conexão: `radar-backend` resolve/conecta, `radar-db` não (fora da backend-net).

Desafios 2 e 3 — Multi-homing / Chaos (queda da rota pública)

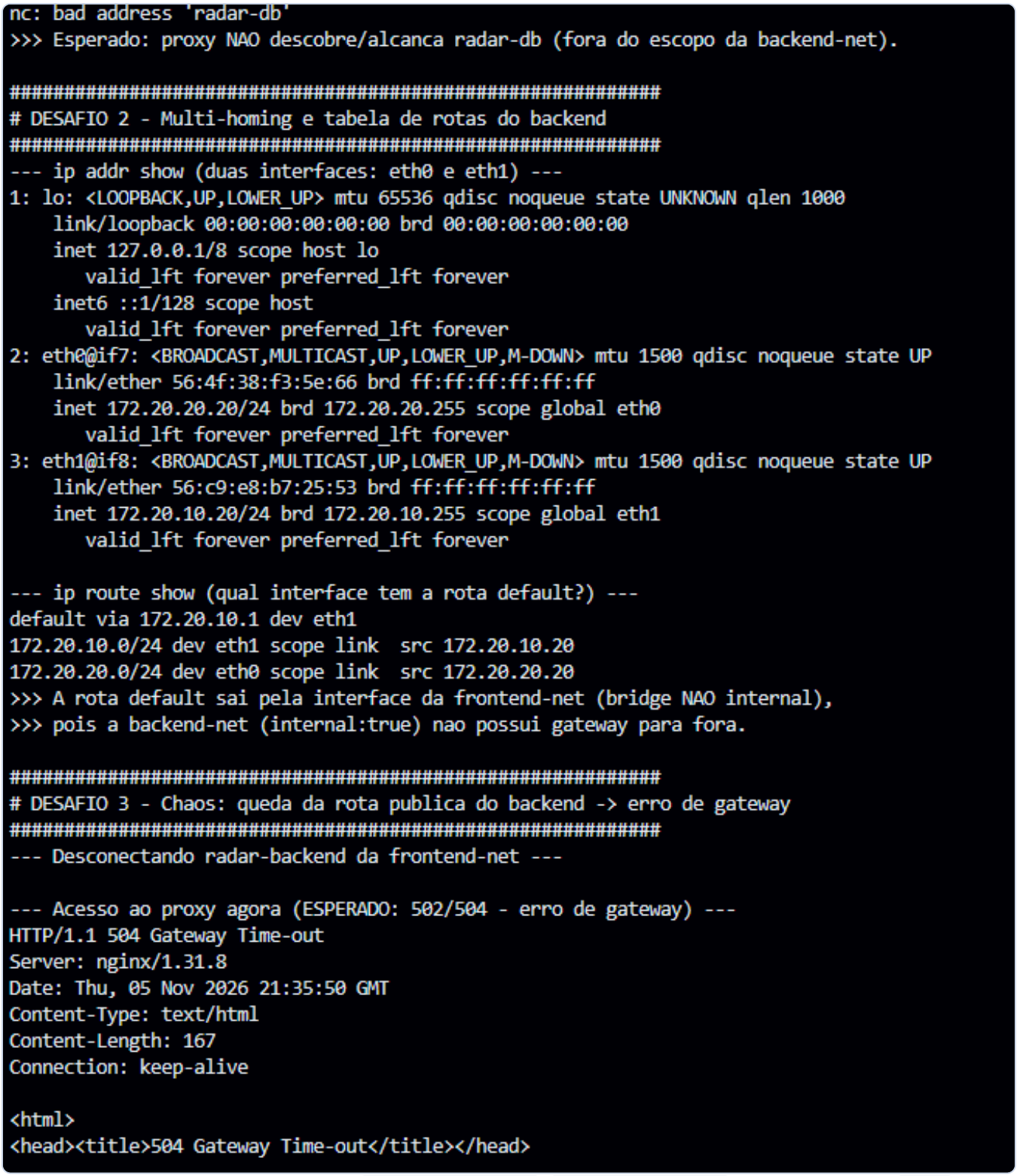


Figura 3.2 — Desafio 2: duas interfaces (`eth0=172.20.20.20` , `eth1=172.20.10.20`) e rota default via `eth1` (frontend-net). Desafio 3: desconexão erro de gateway (`504`).

Desafios 3 e 4 — Recuperação e Bastion Host

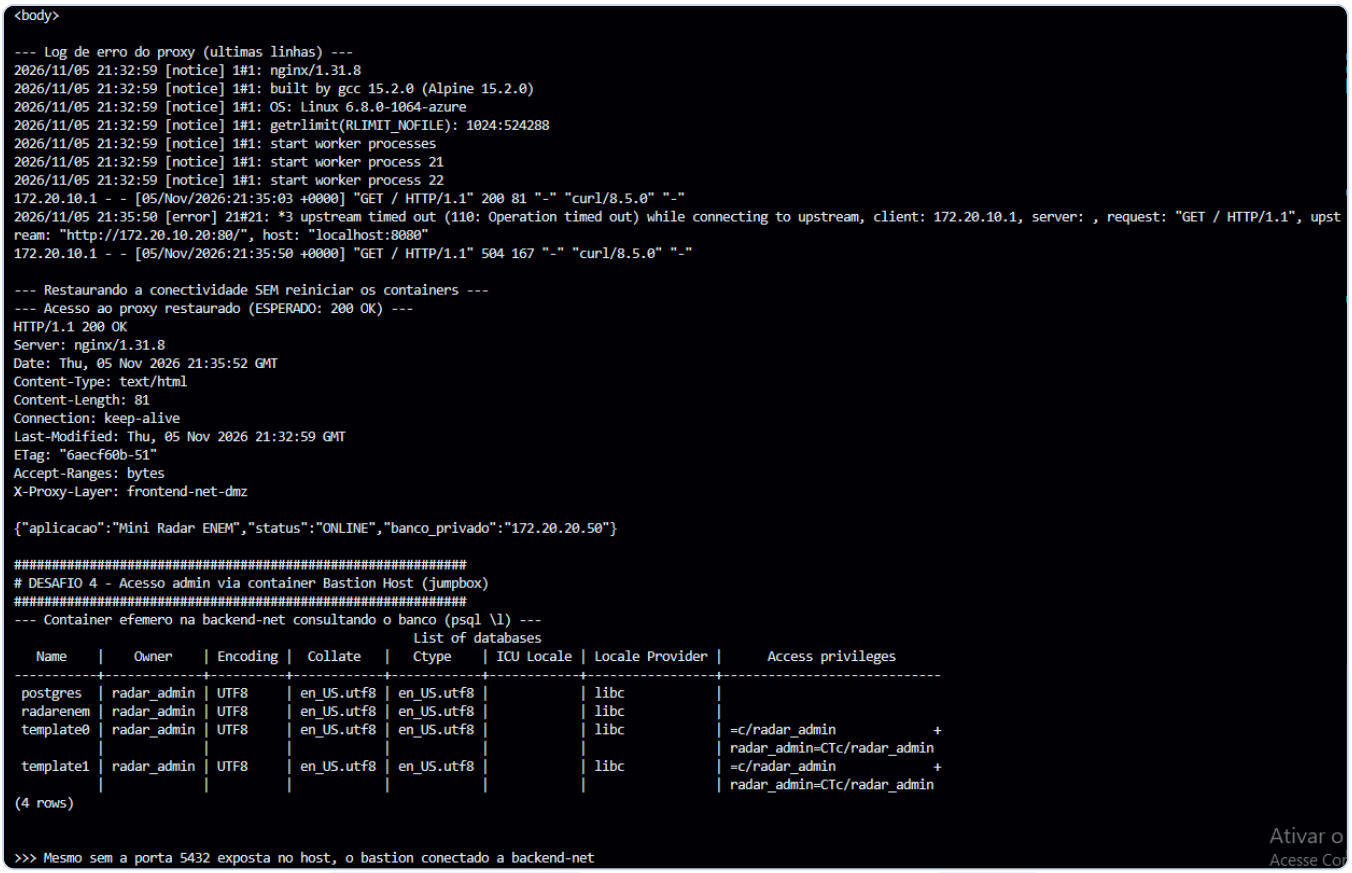


Figura 3.3 — Log do proxy (`upstream timed out`) e restauração sem reiniciar (`200 OK`); Desafio 4: container efêmero na backend-net executa `psql \l` listando o banco `radarenem` .

Desafio 4 — Detalhe do acesso via Bastion Host

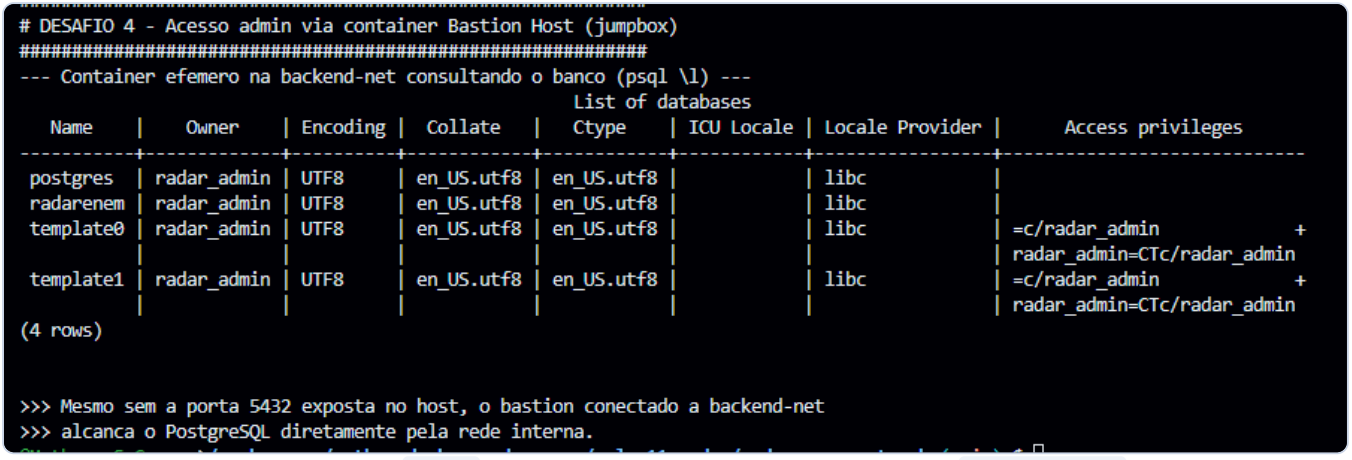


Figura 3.4 — Mesmo sem a porta `5432` exposta no host, o bastion conectado à `backend-net` alcança o PostgreSQL diretamente pela rede interna.

Evidência 4 — Diagrama arquitetural da VPC (Incremento 4)

VPC `10.20.0.0/16` do Radar ENEM com as 3 sub-redes e seus CIDRs, o Internet Gateway, o NAT Gateway e o sentido das setas de tráfego.

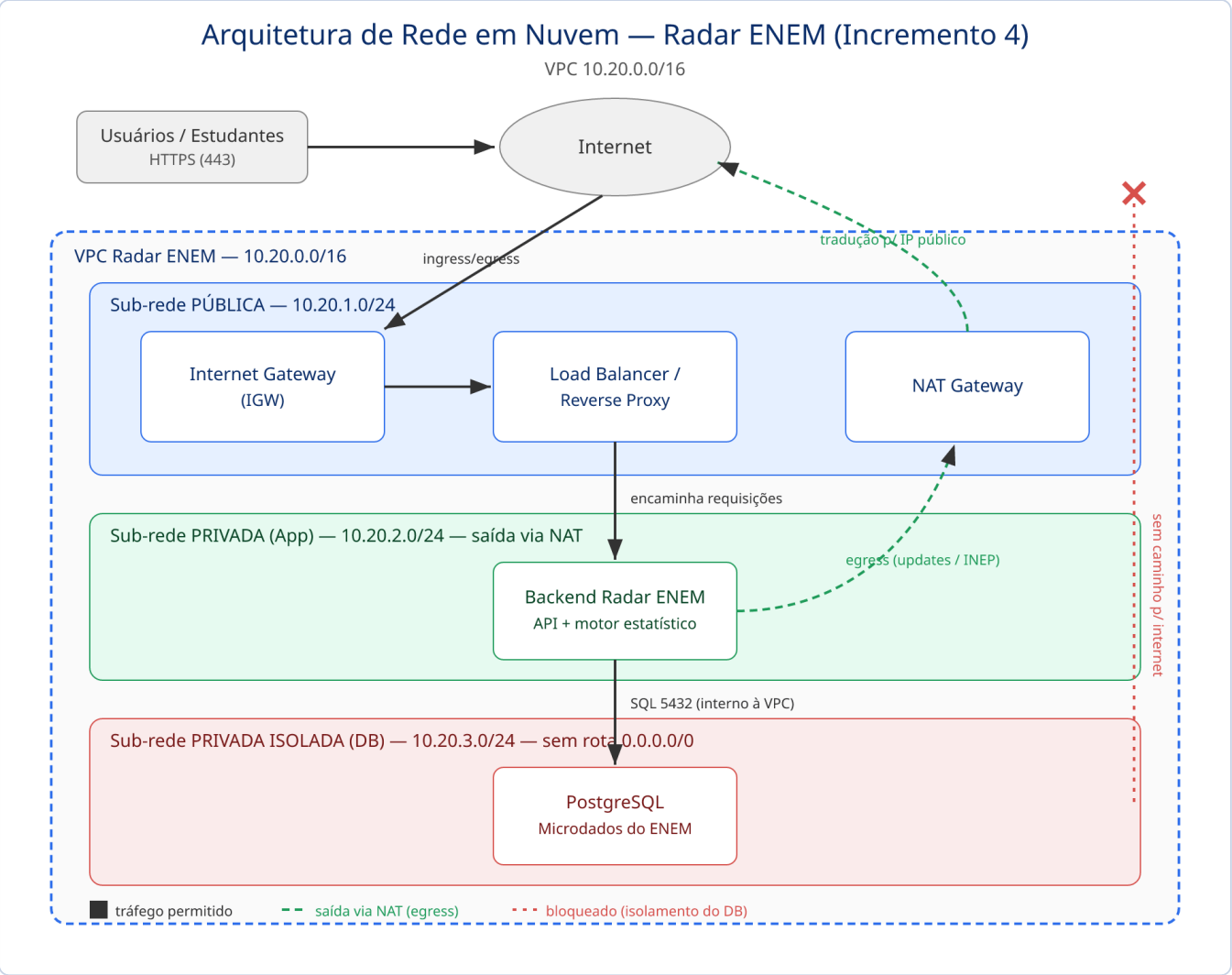


Figura 4 — Pública `10.20.1.0/24` (IGW + LB + NAT GW) · Privada App `10.20.2.0/24` (saída via NAT) · Privada isolada DB `10.20.3.0/24` (sem rota `0.0.0.0/0`).

Relatório Técnico de Análise e Defesa Arquitetural

1. Desafios investigativos (Passo 6)

> As saídas reais de cada desafio estão nas capturas de tela deste documento (seção de evidências). Abaixo, a análise técnica de cada um.

Desafio 1 — Escopo do DNS interno do Docker

Comandos: `cat /etc/resolv.conf`, `nslookup radar-backend`, `nslookup radar-db` (a partir do `radar-proxy`).

- O `/etc/resolv.conf` do `radar-proxy` aponta para o **DNS embutido do Docker em `127.0.0.11`**.
- `nslookup radar-backend` **resolve**, porque proxy e backend compartilham a `frontend-net`.
- `nslookup radar-db` **não resolve** (NXDOMAIN): o banco só existe na `backend-net`, da qual o proxy não participa. O DNS interno do Docker é **escopado por rede** — um container só descobre pelo nome os serviços que estão em alguma rede comum a ele.

Desafio 2 — Multi-homing e tabela de rotas

Comandos: `ip addr show`, `ip route show` (dentro de `radar-backend`).

- `radar-backend` tem **duas interfaces**: `eth0` e `eth1` (uma por rede).
- A **rota default (`default via ...`) sai pela interface da `frontend-net`**. Motivo: a `backend-net` é `internal: true`, ou seja, **não tem gateway para fora**; o Docker só configura gateway/rota padrão pela rede bridge comum (não-internal). Por isso o tráfego "para o mundo" do backend segue pela frontend-net.

Desafio 3 — Chaos: queda da rota pública e erro 502

- Ao executar `docker network disconnect radar-enem-network_frontend-net radar-backend`, o proxy perde o caminho até o backend. O acesso a `localhost:8080` passa a retornar um **erro de gateway** (`502 Bad Gateway` quando a conexão é recusada, ou `504 Gateway Time-out` quando ela estoura o `proxy_connect_timeout` — ambos indicam que o upstream ficou inacessível) e o log do proxy registra algo como *"connect() failed"/"upstream timed out"* para `radar-backend`.
- `docker network connect --ip 172.20.10.20 radar-enem-network_frontend-net radar-backend` **restaura a conectividade sem reiniciar** os containers — o próximo request volta a **200 OK**.

Desafio 4 — Bastion Host (jumpbox)

- Mesmo com a porta `5432` **não publicada** no host, um container efêmero (`--rm`) conectado diretamente à `backend-net` alcança o PostgreSQL em `172.20.20.50` e executa `psql ... -c "\l"`.
- Equivale a um **Bastion Host / AWS Systems Manager Session Manager**: manutenção e auditoria seguras na sub-rede privada, sem abrir o banco para o host ou para a internet.

---

2. Incremento 4 — Arquitetura de rede em nuvem (VPC `10.20.0.0/16`)

2.1. Plano de endereçamento e sub-redes

Componente / Camada	CIDR	IPs (totais / úteis)	Tipo de sub-rede	Finalidade
Entrada / Load Balancer	`10.20.1.0/24`	256 / 251	Pública	Receber tráfego HTTPS externo dos usuários e hospedar o NAT Gateway.
Backend Radar ENEM	`10.20.2.0/24`	256 / 251	Privada (saída via NAT)	Executar a API e o motor estatístico de diagnósticos.
Persistência PostgreSQL	`10.20.3.0/24`	256 / 251	Privada isolada	Armazenar os dados/recortes dos Microdados do ENEM.

2.2. Mapeamento das tabelas de roteamento

Tabela de rotas	Sub-rede	Destino	Próximo salto	Significado
Rota Pública	`10.20.1.0/24`	`10.20.0.0/16` / `0.0.0.0/0`	Local / Internet Gateway (IGW)	Comunicação interna na VPC + rota padrão aberta para a internet.
Rota Privada (App)	`10.20.2.0/24`	`10.20.0.0/16` / `0.0.0.0/0`	Local / NAT Gateway	Bloqueia entrada externa, mas permite saída para baixar pacotes/atualizações.
Rota Privada (DB)	`10.20.3.0/24`	`10.20.0.0/16` / `0.0.0.0/0`	Local / Sem rota (bloqueado)	Isolamento estrito: o banco só se comunica dentro da VPC.

2.3. Correspondência laboratório Docker nuvem

Laboratório Docker	Equivalente na nuvem	Função no Radar ENEM
`frontend-net` (172.20.10.0/24)	Sub-rede pública (rota p/ IGW)	Receber conexões no Reverse Proxy / Load Balancer.
`backend-net` (`internal: true`)	Sub-rede privada isolada (sem `0.0.0.0/0`)	Isolar o PostgreSQL de conexões externas diretas.
DNS interno (`127.0.0.11`)	Route 53 Private Hosted Zone / Azure Private DNS	Resolver nomes de serviços internos sem fixar IPs.
Container efêmero (Desafio 4)	Bastion Host / SSM Session Manager	Manutenção e auditoria seguras na sub-rede privada.

2.4. Diagrama

O diagrama visual da VPC `10.20.0.0/16` (3 sub-redes com seus CIDRs, Internet Gateway, NAT Gateway e sentido das setas de tráfego) consta na seção de evidências deste documento (`evidencias/04_diagrama_vpc.png`).

---

3. Questões discursivas (defesa arquitetural)

3.1. Isolamento em camadas

O `radar-proxy` **não** conseguiu pingar nem resolver `radar-db` porque os dois containers **não compartilham nenhuma rede**: o proxy está só na `frontend-net` e o banco só na `backend-net`. Como o DNS interno do Docker é escopado por rede, o nome `radar-db` sequer é resolvido a partir do proxy; e, mesmo que o proxy tentasse o IP `172.20.20.50`, não há rota/interface ligando-o à `backend-net`. Um invasor que comprometesse o Nginx público ficaria **contido na DMZ**: ele só alcançaria o `radar-backend` (a face que o backend expõe na frontend-net). Para chegar ao banco, seria preciso primeiro comprometer o próprio backend — a única ponte multi-homed — o que cria uma **barreira extra** e **contém o movimento lateral**. A rede interna (`internal: true`), sem gateway para fora, é o que garante que o PostgreSQL não seja acessível diretamente do host nem da DMZ.

3.2. Princípio da exposição mínima e papel do NAT

Colocar o PostgreSQL numa sub-rede **pública**, mesmo com senha forte, viola a **exposição mínima** porque passa a existir um **caminho de rede alcançável de fora** (superfície de ataque exposta). A senha vira a *única* linha de defesa, ficando sujeita a força bruta, exploração de CVEs do serviço, varreduras e vazamentos de credenciais — defesa em profundidade deixa de existir. O correto é manter o banco em sub-rede **privada isolada**, sem rota `0.0.0.0/0`: quem não tem caminho de rede sequer tenta a autenticação.

O **NAT Gateway** entra quando o Backend (em sub-rede privada) precisa de **saída** para a internet — por exemplo, baixar atualizações de pacotes ou arquivos/Microdados do INEP. O NAT permite conexões **iniciadas de dentro para fora** (egress), traduzindo o IP privado para um IP público de saída e mantendo o estado da conexão para o retorno — **sem** criar nenhuma rota de **entrada** não solicitada. Assim o backend consegue atualizar-se e consumir dados externos, mas continua **inalcançável** por conexões iniciadas da internet, preservando o isolamento.